Upstream log: problems the brms-parity loop found in brms, Stan Math and loo

Generated from `docs/brms-parity/upstream/log.json` by `tools/upstream_log.py`

Checked 2026-10-23

Each entry is a place where brms, the Stan Math library as shipped with rstan's StanHeaders, or loo computes something wrong or inexactly, or accepts input it should refuse, found while AutoBayes was compared with brms (`docs/brms-parity/divergences.md`). Nothing here has been posted anywhere.

Every entry has a standalone R script that needs only CRAN brms, rstan and loo (no AutoBayes): it prints what the library gives next to the exact value, which it computes itself from a closed form or a stable log-space formula, and exits with status 1 while the problem is present. Densities and gradients come from `brm(... , chains = 0)` with `rstan::log_prob` and `grad_log_prob` (the prior's part removed by evaluating the same compiled model with its `prior_only` flag set); predictions from fixed-parameter runs. Nothing samples.

The fixes are described from the mathematics. brms's, loo's and Stan's source code was not read; where a cause is inferred from outputs it is said so. Known-upstream status comes from read-only searches of brms's NEWS and the paul-buerkner/brms, stan-dev/math and stan-dev/loo issue trackers.

Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, loo 2.9.0, R 4.5.2.

Entries: 30 reproduced, 1 not a bug after all.

1 All entries

Id	Component	Problem	Kind	Status	Known upstream
U01	brms	hurdle_negbinomial `posterior_predict()` does not draw the positive part from the zero-truncated negative binomial	wrong result	reproduced	not found
U02	brms	`posterior_epred()` of a count response with `trunc(lb = )` starts the support at lb + 1 and cuts it off near 3 mu	wrong result	reproduced	#1923 (lb + 1 fixed in development)
U03	brms	cumulative("cloglog") forms its upper categories' probabilities by subtraction; the log density loses them and is -Inf beyond d of about 3.6	inexact numerics	reproduced	not found (related: #1951)
U04	brms	The Beta family forms `1 - mu` by subtraction: digits lost, and `beta_lpdf` rejects the draw once mu rounds to 1	inexact numerics	reproduced	not found
U05	brms	Bernoulli and binomial cloglog likelihoods form a failure's `log(1 - theta)` from `theta = inv_cloglog(eta)`	inexact numerics	reproduced	not found
U06	brms	Bernoulli and binomial probit likelihoods are -Inf where Stan's Phi saturates (eta above 8.25 for a failure, below -37.5 for a success)	inexact numerics	reproduced	not found
U07	brms	Hurdle and zero-inflated Poisson densities and the hurdle mean break at extreme linear predictors (+Inf, infinite or NaN gradients, NaN mean)	inexact numerics	reproduced	not found
U08	brms	`log_lik()` is -Inf, +Inf or NaN (or inexact) at extreme but finite linear predictors	inexact numerics	reproduced	related: #1951 (ordinal, open)
U09	brms	Cumulative `posterior_epred()` forms category probabilities by subtraction, losing digits between close thresholds	inexact numerics	reproduced	not found
U10	brms	Truncated Gaussian `posterior_epred()` loses its digits in the upper tail and is Inf or NaN beyond about 8.3 SD	inexact numerics	reproduced	not found

Id	Component	Problem	Kind	Status	Known upstream
U11	brms	Interval censoring takes the mass as `log_diff_exp` of two log CDFs, which loses every digit in the upper tail	inexact numerics	reproduced	related: #1956 (open)
U12	brms	gen_extreme_value `posterior_epred()` is finite for xi > 1, NaN at xi = 0 and inexact near 0	wrong result	reproduced	not found
U13	brms	`gr(g, by = )`: the SD prior's truncation constant is added once per coefficient, not once per by level	wrong result	reproduced	not found
U14	brms	Grouping factor `g:h` joins labels with `"_"`, silently merging combinations whose joined labels coincide	wrong result	reproduced	not found
U15	brms	A non-linear formula's integer division is Stan's in the model and R's in the predictions: `(1 / 2) * a` fits mu = 0 and predicts a / 2	wrong result	reproduced	not found
U16	brms	Functions Stan does not have (`plogis()`, and `gp()`, `mo()`, `ar()` written in the expression) pass into a non-linear model's program unchecked	missing validation	reproduced	not found
U17	brms	Under identity and inverse links a non-positive mean is returned by `posterior_epred()` without a word, and `posterior_predict()` draws NaN	missing validation	reproduced	not found
U18	brms	asym_laplace accepts a fixed quantile of 0 or 1, where the density is 0 everywhere	missing validation	reproduced	not found
U19	brms	The default length-scale prior of `gp()` is solved to a tolerance coarser than the 6 decimals it is printed and written with	inexact numerics	reproduced	not found
U20	brms	Approximate GP `gp(x, k = )`: NaN gradient wherever the spectral density underflows (every lscale above about 1 with k = 30)	inexact numerics	reproduced	not found

Id	Component	Problem	Kind	Status	Known upstream
U21	brms	Priors on probability parameters (zi, hu, zoi, coi) are renormalised on (0, 1): divergence row D29 is not what brms does	wrong result	not a bug after all	n/a
U22	Stan Math	Stan's `normal_lccdf` loses its value in the upper tail (-Inf from z of about 8.3), so brms's right-censored rows are -Inf and truncated rows +Inf	wrong result	reproduced	math #1985, fixed in Math 5.4.0
U23	Stan Math	Stan's `normal_lcdf` / `std_normal_lcdf` derivative is inexact (up to 2.3e-5 relative), and so are brms's censored Gaussian gradients	inexact numerics	reproduced	related: math PR #3406 (open)
U24	Stan Math	cumulative("probit") through Stan's `ordered_probit_lpmf`: exact log density, gradient off by up to 1.2e-5 relative	inexact numerics	reproduced	related: math PR #3406 (open)
U25	Stan Math	Stan's `gamma_lccdf` derivative in the shape is inexact (up to 1.2e-2 relative), and so are brms's right-censored Gamma gradients	inexact numerics	reproduced	math #2006 (open)
U26	Stan Math	Stan's `von_mises_lpdf` overflows (`cyl_bessel_i` numeric overflow) for kappa above about 713.9	wrong result	reproduced	math #3035, probably fixed in Math 4.9.0
U27	Stan Math	Stan's `skew_normal_lpdf` and `exp_mod_normal_lpdf` are -Inf once their normal-CDF factor underflows (argument below about -37.5)	inexact numerics	reproduced	related: math PR #3406 (open)
U28	loo	loo's `stacking_weights()` stops short of the optimum (weights off by up to 3.7e-4, no exact zeros)	inexact numerics	reproduced	loo #119, PR #412 (open)
U29	loo	`loo_compare()` compares a PSIS-LOO result with a WAIC result without an error or a warning	missing validation	reproduced	not found
U30	loo	loo's MCSE of an observation's elpd_loo is NaN once that elpd is below about -372	inexact numerics	reproduced	related: loo #245, PR #395

Id	Component	Problem	Kind	Status	Known upstream
U31	brms	A one-row model with `trunc()` (a per-row or a constant bound) cannot be built: rstan stops at data initialisation	wrong result	reproduced	related: #545 (closed 2018)

2 The entries

2.1 U01: hurdle_negbinomial `posterior_predict()` does not draw the positive part from the zero-truncated negative binomial

Where

brms: posterior_predict(), hurdle_negbinomial(). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

D03 (#65); divergence D86. Recorded in `tests/zero_hurdle_probes`; reproduced independently in the Tier D review.

What

The share of zeros is right (`hu`), but the positive draws are too concentrated at 1 and their mean is too small,

so the draws contradict brms's own `posterior_epred()`, which reports the zero-truncated mean.

Observed

At `mu = 0.5, shape = 0.5, hu = 0.05` (fixed-parameter run, 100000 draws, 95122 positive): P(y = 1 | y > 0) = 0.8164 and E[y | y > 0] = 1.2709. At `mu = 3, shape = 2`: 0.2608 and 3.1557. At `mu = 0.05, shape = 0.2`: 0.9774 and 1.0246. `posterior_epred()` gives 1.621751442, 3.392857143 and 1.088263927, the exact values.

Expected

The zero-truncated negative binomial: P(y = 1 | y > 0) = 0.6036, 0.2286, 0.8764 and E[y | y > 0] = `mu / (1 - NB(0))` = 1.7071, 3.5714, 1.1455 (between 24 and 134 standard errors from brms's draws).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U01.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U01.out`.

Fix

Draw the positive part by inverting the survival function on its own range: `v ~ Uniform(0, 1 - NB(0))`, `y = qnbinom(v, size = shape, mu = mu, lower.tail = FALSE)`, which is exactly the zero-truncated distribution and has no rejection loop when NB(0) is near 1.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.2 U02: `posterior_epred()` of a count response with `trunc(lb = )` starts the support at lb + 1 and cuts it off near 3 mu

Where

brms: posterior_epred(), fitted(), trunc() for negbinomial (and the other count families). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

E04 (#77); divergence D121.

What

The likelihood of `y | trunc(lb = 1)` is normalised by P(Y >= 1), but the expected value is the mean over 2..M with a finite M, so it is neither E[Y | Y >= 1] nor E[Y | Y >= 2].

Observed

Negative binomial `y | trunc(lb = 1) ~ 1`: at `mu = 1.5, shape = 1` brms gives 2.673469388 = the mean over 2..4; at `mu = 134, shape = 0.6` it gives 102.6108362 = the mean over 2..402, whatever the data's largest count (5 or 300). On the frozen case `negbinomial_truncated` (point 7, row 27) 106.236, the mean over 2..394.

Expected

E[Y | Y >= 1] = `mu / (1 - NB(0))`: 2.5 and 139.4172675 (106.236 should be 133.95). The likelihood (checked with `log_prob`) is normalised by P(Y >= 1), so the expected value must be that distribution's.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U02.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U02.out`.

Fix

For a lower bound alone use the closed form `E[Y | Y >= lb] = (mu - sum_{k < lb} k P(k)) / (1 - F(lb - 1))`, which needs no truncation of the support; with both bounds sum `k P(k)` over `lb..ub` and divide by `F(ub) - F(lb - 1)`.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Known: the lb + 1 start is brms issue #1923, fixed by PR #1926 in the development version.
https://github.com/paul-buerkner/brms/issues/1923 (closed 2026-09-01): `posterior_epred()` drops

the lower bound for truncated discrete models
https://github.com/paul-buerkner/brms/pull/1926 (merged 2026-09-01): Include the lower bound in truncated discrete posterior_epred
Fixed in: the development version (2.23.2, NEWS: Include the lower bound when computing `posterior_epred` for truncated discrete models); CRAN still has 2.23.0 (checked 2026-10-21).
The finite upper end of the support (about 3 mu here) for a lower bound alone is not mentioned in #1923 and was not checked against the development version.

2.3 U03: cumulative("cloglog") forms its upper categories' probabilities by subtraction; the log density loses them and is -Inf beyond d of about 3.6

Where
brms: cumulative(link = "cloglog") (generated Stan code). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
E06 (#71); divergence D132. The link was accepted on a narrower case (#79); the frozen case `cumulative_cloglog` keeps its G2 gates expected to fail.

What
The last category's probability is `1 - inv_cloglog(d)` and the others `inv_cloglog(d_k) - inv_cloglog(d_{k-1})`, formed by subtraction of numbers near 1, so an improbable category keeps only a few digits, and none once `inv_cloglog(d)` rounds to 1.

Observed
Intercept-only model, one row per category, thresholds (-1, 3, 3.375) (P = 0.308, 0.692, 1.89e-9, 2.03e-13): log likelihood -50.85601389939049, gradient off by up to 1.0e-4 relative. Thresholds (-1, 3, 4) (P(y = 4) = 1.94e-24): log likelihood -Inf, gradient -Inf. On the frozen case `cumulative_cloglog` the log density is 5.9e-7 relative off (1.45e-4 absolute) and the gradient up to 1.3e-4.

Expected
-50.85611467237852 (relative error 2e-6) and -76.22987349665763, with finite gradients; at the control thresholds (-1, 0, 1) brms agrees to 1.4e-16.

Reproduce
`Rscript docs/brms-parity/upstream/repro/U03.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U03.out`.

Fix
Evaluate every category on the log scale from the survival function `S(d) = 1 - inv_cloglog(d) = exp(-exp(d))`, with `d_k = tau_k - eta`: `log P(y = 1) = log1m_exp(-exp(d_1))`, `log P(y = K) = -exp(d_{K-1})`, and `log P(y = k) = -exp(d_{k-1}) + log1m_exp(-(exp(d_k) - exp(d_{k-1})))` (that is, `log_diff_exp` of the two log survival values).

Belongs in
https://github.com/paul-buerkner/brms

Known upstream
Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).
Related but not the same: open PR #1951 (Evaluate ordinal log_lik and log_softmax() on the log scale) is about brms's R-side log_lik, not the Stan code.

2.4 U04: The Beta family forms `1 - mu` by subtraction: digits lost, and `beta_lpdf` rejects the draw once mu rounds to 1

Where
brms: Beta(link = "logit" | "probit" | "cloglog") (generated Stan code). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
A11, E06 (#32, #71); divergence D13, D132.

What

The second shape is `(1 - mu) * phi` with `1 - mu` by subtraction: it loses `eps / (1 - mu)` relative, and is exactly 0 above eta of about 37 (logit), 8.25 (probit; Stan's Phi is 1 there) and 3.6 (cloglog), where Stan's `beta_lpdf` raises an error and the draw is rejected.

Observed

y = 0.5, phi = 5: logit eta = 30 gives -30.47102418467954 (relative error 3.3e-5), eta = 40 an error (`beta_lpdf: Second shape parameter[1] is 0`); probit eta = 8 gives -35.41504472969478 (1.9e-3), eta = 9 the error; cloglog eta = 3.4 gives -30.4337238232663 (1.2e-5), eta = 3.7 the error.

Expected

-30.47000362924486 and -40.47000362924573 (logit), -35.48344078916028 and -44.09815274257785 (probit), -30.43410367664173 and -40.91730798931313 (cloglog); at the control points (eta = 1, 1, 0.5) brms agrees to 4e-15.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U04.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U04.out`.

Fix

Take `1 - mu` from the link's other tail instead of subtracting: `inv_logit(-eta)`, `Phi(-eta)`, `exp(-exp(eta))`, and pass `beta_lpdf(y | mu * phi, q * phi)` with that `q`. (Where even `q` underflows, about eta = 37.5 for probit, the shape needs `lgamma` from `log q`.)

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.5 U05: Bernoulli and binomial cloglog likelihoods form a failure's `log(1 - theta)` from `theta = inv_cloglog(eta)`

Where

brms: bernoulli(link = "cloglog"), binomial(link = "cloglog") (generated Stan code). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

A12 (#36); divergence D18. Accepted with a 50-digit reference for the gradient (P6, #47).

What

`bernoulli_lpmf(y | inv_cloglog(eta))` (and the binomial's) differentiates and evaluates `log(1 - theta)` after `theta` has rounded: the gradient loses about `eps * exp(exp(eta))` relative, and the log density of a failure is -Inf once theta rounds to 1 (eta of about 3.6), where the exact value is `-exp(eta)`.

Observed

Bernoulli y = 0: eta = 3 log likelihood -20.08553689560981 (1.4e-9), gradient -20.08553636927165 (2.8e-8); eta = 3.5: -33.12588265703288 (3.1e-4), gradient -33.46267700016426 (1%); eta = 4: -Inf, gradient +Inf. Binomial 1 of 3: eta = 3.5 -65.15315302539764 (3.2e-4), eta = 4 -Inf. On the frozen case `bernoulli_cloglog` one gradient entry is 3.8e-7 off.

Expected

Bernoulli: `-exp(eta)` = -20.08553692318767, -33.11545195869231, -54.59815003314424 (gradients equal). Binomial: `log 3 + log1m_exp(-exp(eta)) - 2 exp(eta)` = -65.13229162871652 and -108.0976877776204. At eta = 0 brms agrees to 4e-16.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U05.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U05.out`.

Fix

Write the likelihood on the log scale: a success adds `log1m_exp(-exp(eta))`, a failure `-exp(eta)`; the binomial `y log1m_exp(-exp(eta)) - (n - y) exp(eta)` plus the log binomial coefficient.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.6 U06: Bernoulli and binomial probit likelihoods are -Inf where Stan's Phi saturates (eta above 8.25 for a failure, below -37.5 for a success)

Where

brms: bernoulli(link = "probit"), binomial(link = "probit") (generated Stan code). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

A12 (#36); divergence D17.

What

`bernoulli_lpmf(y | Phi(eta))`: Stan's `Phi` is exactly 1 above 8.25 and 0 below -37.5, so `log(1 - theta)` of a failure is -Inf above 8.25 (and loses digits from about 5), and `log(theta)` of a success is -Inf below -37.5, with infinite or NaN gradients.

Observed

Bernoulli y = 0: eta = 5 -15.06499839422134 (1.5e-11), eta = 8 -34.94504110044905 (2e-3; gradient 6.6% off), eta = 8.5 and 9 -Inf. Bernoulli y = 1: eta = -38 -Inf (gradient +Inf), eta = -40 -Inf (gradient NaN). Binomial 0 of 3: eta = 8 -104.8351233013471, eta = 9 -Inf.

Expected

`log Phi(-eta)` = -15.06499839398873, -35.01343715991455, -39.19739642821767, -43.62814911333211; `log Phi(eta)` = -726.5572160188201 and -804.6084420137538; binomial -105.0403114797437 and -130.8844473399963. At eta = 0 brms agrees to 2e-16.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U06.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U06.out`.

Fix

Use the log CDF: a success adds `std_normal_lcdf(eta)`, a failure `std_normal_lcdf(-eta)` (binomial: `y std_normal_lcdf(eta) + (n - y) std_normal_lcdf(-eta)` plus the log coefficient). Its value is exact in double precision down to -40 in these probes; its derivative carries U23's error until Stan Math fixes that.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.7 U07: Hurdle and zero-inflated Poisson densities and the hurdle mean break at extreme linear predictors (+Inf, infinite or NaN gradients, NaN mean)

Where

brms: hurdle_poisson(), zero_inflated_poisson() (brms's Stan functions), posterior_epred() of hurdle_poisson(). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

B06 (#39); divergence D28.

What

A hurdle y > 0 row subtracts `log1m_exp(-exp(eta))`, which is `log(0)` once `exp(eta)` underflows, so the row's log density is +Inf; a zero-inflated zero's gradient is NaN once `exp(eta)` overflows; the hurdle mean `mu / (1 - exp(-mu)) * (1 - hu)` is 0/0 where mu underflows.

Observed

hurdle_poisson, y = 1, hu = 0.3: eta = -740 log likelihood -0.3592542790478319 with gradient -Inf; eta = -750 +Inf with gradient NaN. zero_inflated_poisson, y = 0, zi = 0.3, eta = 710: value -1.203972804325936 (right) with gradient NaN. `posterior_epred()` at eta = -750: NaN.

Expected

y = 1: `log(1 - hu) - mu/2` = -0.3566749439387422 at both points with gradient about 0; the zero-inflated zero `log(zi)` with gradient (0, 1 - zi); the hurdle mean's limit `1 - hu` = 0.7. At eta = 0 brms agrees to 2e-16.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U07.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U07.out`.

Fix

Evaluate `log(1 - exp(-mu))`, `mu = exp(eta)`, as `eta + log1p(-mu / 2)` below eta of about -20 (and `log(-expm1(-mu))` above), so it never forms `log(0)`; the zero-inflated zero as `log_sum_exp(log(zi), log1m(zi) - exp(eta))`, whose gradient is 0 where the second term is -Inf; the hurdle mean as `(1 - hu) exp(eta - log(1 - exp(-mu)))` with the same stable form (its limit 1 - hu).

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.8 U08: `log_lik()` is -Inf, +Inf or NaN (or inexact) at extreme but finite linear predictors

Where

brms: log_lik(), loo(), waic() (R-side log-likelihoods of bernoulli, binomial, hurdle_poisson, cumulative, mixture). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

C07 (#49); divergence D49. The mixture case was described from brms's source in the row; here it is shown from output.

What

The pointwise log-likelihood is evaluated in R from probabilities that round to 0 or 1, and a mixture's from a sum of component densities without factoring the largest out.

Observed

Bernoulli logit y = 0: eta = 35.07 gives -34.94504110044905 (3.6e-3), eta = 40 -Inf. Binomial 0 of 3: eta = 27.2 -81.60011489079963 (1.4e-6), eta = 40 -Inf. hurdle_poisson y = 1: eta = -26.9 -0.3566877935318438 (3.6e-5), eta = -60 +Inf. cumulative logit (-1, 0, 1): y = 4 at eta = -640 -Inf, y = 2 at eta = -640 NaN (with brms's warning that NAs were found). Two-Gaussian mixture at y = 40: -Inf.

Expected

-35.07 and -40; -81.60000000000461 and -120; -0.3566749439397725 and -0.3566749439387351; -641 and -639.4586751453871; -762.1120857137646. Control rows (eta = 0, y = 0) agree exactly.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U08.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U08.out`.

Fix

Evaluate each row on the log scale, as the Stan model does: logit `y eta - log1p_exp(eta)` (binomial `y eta - n log1p_exp(eta)` plus the coefficient); the hurdle as in U07; ordinal categories from `log_inv_logit_diff` or `plogis(., log.p = TRUE)` of the tail; a mixture as `log_sum_exp` over `log theta_k + log p_k`.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Partly related work upstream, not this problem as such.

https://github.com/paul-buerkner/brms/pull/1951 (open PR): Evaluate ordinal log_lik and log_softmax() on the log scale
brms's development NEWS adds a warning when `log_lik` returns infinite values (#1899), which reports the problem without removing it. The Bernoulli, binomial, hurdle and mixture rows were not found in the tracker.

2.9 U09: Cumulative `posterior_epred()` forms category probabilities by subtraction, losing digits between close thresholds

Where
brms: posterior_epred() of cumulative() (R side). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
B05 (#41); divergence D39.

What
P(y = k) is `plogis(tau_k - eta) - plogis(tau_{k-1} - eta)`, so a category between two close thresholds, or one small next to its neighbours in the upper tail, keeps only some of its digits.

Observed
Thresholds (0, 1e-9, 2): P(y = 2) = 2.500000206850927e-10 (8.3e-8 relative); (0, 1e-6, 2): 1.4e-10; (-1, 20, 20.001): P(y = 3) = 2.06012984449444e-12 (3.1e-6) and P(y = 4) = 2.059093562323255e-09 (3.3e-8).

Expected
2.5e-10, 2.499999999999791e-07, 2.060123380581377e-12 and 2.059093494809622e-09; at (-1, 0, 1) brms agrees to 1.2e-16.

Reproduce
`Rscript docs/brms-parity/upstream/repro/U09.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U09.out`.

Fix
`plogis(b) - plogis(a) = plogis(a) * plogis(-b) * expm1(b - a)` for `b > a` (no difference formed), or the exponential of the `log_inv_logit_diff` form the Stan code uses; the last category as `plogis(eta - tau_{K-1})`.

Belongs in
https://github.com/paul-buerkner/brms

Known upstream
Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.10 U10: Truncated Gaussian `posterior_epred()` loses its digits in the upper tail and is Inf or NaN beyond about 8.3 SD

Where
brms: posterior_epred() of gaussian() with trunc() (R side). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
B02 (#40); divergence D25. Accepted with a 50-digit reference for these expected values (P6, #47).

What
The truncated mean is formed with `pnorm(ub) - pnorm(lb)` by subtraction, so a bound several SD above the location loses the denominator's digits.

Observed
mu = 0, sigma = 1, `y | trunc(lb = L)`: L = 6.17 gives 6.324502577466615 (4.5e-8), L = 7.5 7.639864143170588 (1.4e-3), L = 8.2 9.005738961208227 (8.3%), L = 9 Inf, L = 40 NaN. On the frozen case `gauss_truncated` up to 1.05e-3 at 6 of 656 entries.

Expected

`mu + sigma phi(z) / (1 - Phi(z))`: 6.32450286358622, 7.628966391103763, 8.318567530824906, 9.108523105002844, 40.02496884720634; at L = 0 and 3 brms agrees to 6e-15.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U10.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U10.out`.

Fix

Compute the ratio on the log scale: for a lower bound alone `mu + sigma exp(dnorm(a, log = TRUE) - pnorm(a, lower.tail = FALSE, log.p = TRUE))`, `a = (lb - mu) / sigma`; with two bounds take `Phi(b) - Phi(a)` from the tail both lie in (`pnorm(a, lower.tail = FALSE) - pnorm(b, lower.tail = FALSE)` when `a > 0`) or in log space.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.11 U11: Interval censoring takes the mass as `log_diff_exp` of two log CDFs, which loses every digit in the upper tail

Where

brms: cens(c, y2) interval rows (generated Stan code), weibull_lcdf, gamma_lcdf. Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

E04 (#77); divergence D120. E04's probes used rstan::expose_stan_functions; here the same numbers come from brms models. `weibull_censored_interval_post` could not be frozen on brms's default point box for this reason.

What

An interval row (y, y2] gets `log_diff_exp(lcdf(y2), lcdf(y))`; in the upper tail both log CDFs are within a few eps of 0 (Stan 2.32's `weibull_lcdf(5 | 3, 1)` is 0 where the value is -exp(-125)), so the difference has no digits left. brms's own right-censored rows use the `lccdf` and are exact there.

Observed

Weibull (scale 1) (5, 6] at shape 3: -Inf; (6, 8] at shape 2: -36.04365338911715 (= log(eps)); Gamma(2, rate 1) (40, 50]: -36.04365338911715.

Expected

-125 (the mass is exp(-125) - exp(-216)), -36.00000000000069 and -36.28648440797372, from the survival functions; the right-censored Weibull row at 5 (brms -125.0000000000001) and the controls (0.5, 1] and (1, 2] agree to 6e-16.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U11.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U11.out`.

Fix

Choose the tail per row: when `F(y) > 1/2` use `log_diff_exp(lccdf(y), lccdf(y2))`, otherwise `log_diff_exp(lcdf(y2), lcdf(y))`. (Stan Math could also return the upper tail of its lcdfs accurately, e.g. `weibull_lcdf(y) = log1m_exp(-(y / sigma)^alpha)`, `gamma_lcdf = log1m(gamma_q)` above the median, but the tail choice in brms suffices for any version.)

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

The same form for two-sided truncation is reported (open); interval censoring itself was not found.

https://github.com/paul-buerkner/brms/issues/1956 (open): Two-sided truncation in the Stan code loses the normalizer in the upper tail

https://github.com/stan-dev/math/pull/3037 (merged 2024-03-26): Make Weibull cdf & lcdf more ro-

bust, handle y = 0 inputs
https://github.com/stan-dev/math/pull/3408 (open PR): Incomplete gamma gradient roots and log-space cdfs for six distributions
Stan Math PR #3037 (first in Math 4.9.0, newer than StanHeaders 2.32.10) may make `weibull_lcdf` exact in its upper tail; not verified, since nothing newer could be installed.

2.12 U12: gen_extreme_value `posterior_epred()` is finite for xi > 1, NaN at xi = 0 and inexact near 0

Where
brms: posterior_epred() of gen_extreme_value(). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
E02 (#75); divergence D114. brms 2.23 warns that the family is deprecated.

What
The mean is evaluated as `mu + sigma (gamma(1 - xi) - 1) / xi` for every xi: for 1 < xi < 2 `gamma(1 - xi)` is negative and the 'mean' a finite negative number, though the mean is infinite for xi >= 1; at xi = 0 it is 0/0; near 0 `1 - xi` rounds away xi's low digits.

Observed
mu = 0.1, sigma = 1, xi fixed in the formula: xi = 1.5 gives -2.929938467874022, xi = 0 NaN, xi = 1e-10 0.6772160527328651 (5.7e-7 relative). With xi estimated (D114's probe) -1.9e-3 at xi = 9.82.

Expected
+Inf (no finite mean) for xi >= 1; `mu + sigma * euler` = 0.6772156649015323 at xi = 0; 0.6772156650004378 at xi = 1e-10; at the control xi = -0.5 brms is exact (0.3275461490944839).

Reproduce
`Rscript docs/brms-parity/upstream/repro/U12.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U12.out`.

Fix
Return +Inf (or NA with a warning) for xi >= 1; otherwise `mu + sigma * expm1(lgamma(1 - xi)) / xi` with `lgamma(1 - xi)` from its series `euler xi + sum_{k >= 2} zeta(k) xi^k / k` when |xi| is small, whose limit at xi = 0 is `mu + sigma * euler`.

Belongs in
https://github.com/paul-buerkner/brms

Known upstream
Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.13 U13: `gr(g, by = )`: the SD prior's truncation constant is added once per coefficient, not once per by level

Where
brms: (1 | gr(g, by = gby)) (generated Stan code: lprior). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
D08 (#59); divergence D105. AutoBayes reproduces brms's constant (`1|g_sigma_by_normalisation`) to match it.

What
Each of the Nby SD parameters has a half-normal (half-t, half-Cauchy) prior truncated at 0, but `lprior` adds `-log P(sd > 0) = log 2` only once per coefficient, so it is `(Nby - 1) log 2` too small. Sampling is unaffected; bridge sampling's marginal likelihood (and so Bayes factors) is off by that factor.

Observed
gby with 3 levels, sd ~ normal(0, 1): lprior -5.96256441474146.

Expected
-4.57627005362157, the difference -1.38629436111989 = -2 log 2; with gby of one level (the same compiled model) brms is exact (difference 4e-16).

Reproduce
`Rscript docs/brms-parity/upstream/repro/U13.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U13.out`.

Fix
Multiply the truncation term by the number of SD elements, `M x Nby`, so each SD parameter's prior is normalised on (0, Inf).

Belongs in
https://github.com/paul-buerkner/brms

Known upstream
Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.14 U14: Grouping factor `g:h` joins labels with "__", silently merging combinations whose joined labels coincide

Where
brms: (1 | g:h), make_standata(). Wrong result; reproduced. Versions: brms 2.23.0, R 4.5.2.

Found by
A03 (#31); divergence D12.

What
(g = "a_b", h = "c") and (g = "a", h = "b_c") both become "a_b_c" and so one group level, without a warning.

Observed
Three distinct (g, h) combinations: `N_1 = 2` levels, `J_1 = 1 1 2 1 1 2` (rows of (a_b, c) and (a, b_c) in level 1).

Expected
`N_1 = 3`, the two combinations in different levels.

Reproduce
`Rscript docs/brms-parity/upstream/repro/U14.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U14.out`.

Fix
Code the combined factor from the pair of levels (e.g. the pair's unique index, or `interaction(g, h, drop = TRUE)` with a separator that cannot occur in labels), or refuse joined labels that collide.

Belongs in
https://github.com/paul-buerkner/brms

Known upstream
Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.15 U15: A non-linear formula's integer division is Stan's in the model and R's in the predictions: `(1 / 2) * a` fits mu = 0 and predicts a / 2

Where
brms: bf(..., nl = TRUE), posterior_epred(). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by
C02 (#52); divergence D61.

What
The expression is written into the Stan program as it stands, where `1 / 2` of two integer literals is integer division (0), and evaluated in R for predictions, where it is 0.5.

Observed

`bf(y ~ (1 / 2) * a, a ~ 1, nl = TRUE)`, y = 1, a = 2, sigma = 1: the model's log likelihood is -1.418938533204673 (that of mu = 0); `posterior_epred()` gives 1 (mu = 0.5 a).

Expected

One meaning in both: as R reads the formula, mu = 1 and the log likelihood -0.9189385332046728.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U15.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U15.out`.

Fix

Write numeric literals into the Stan program as reals (`1` as `1.0`), so that Stan divides as R does, or refuse an integer division in a non-linear formula.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.16 U16: Functions Stan does not have (`plogis()`, and `gp()`, `mo()`, `ar()` written in the expression) pass into a non-linear model's program unchecked

Where

brms: bf(..., nl = TRUE), make_stancode(), brm(). Missing validation; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

C02, C04 (#52, #53); divergence D61, D70.

What

brms writes the expression into the program without checking its functions, so the user gets a Stan semantic error instead of a message from brms (and `plogis()`, which brms's R-side predictions evaluate, is not translated to `inv_logit`).

Observed

`y ~ plogis(a * x)`, `y ~ a * gp(x)`, `y ~ a * mo(z)`, `y ~ a + ar(t)`: brms generates a program; Stan's compiler: "A returning function was expected but an undeclared identifier 'plogis' was supplied" (and 'gp', 'mo', 'ar').

Expected

A refusal from brms naming the function and what to write instead (or a translation of `plogis` to `inv_logit`); the control `y ~ inv_logit(a * x)` compiles.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U16.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U16.out`.

Fix

Check each function name of a non-linear formula against the functions Stan and brms's Stan code provide before generating code, translating R's `plogis`, `qlogis`, `pnorm` to `inv_logit`, `logit`, `Phi` or refusing with a message, and refuse `gp()`, `mo()`, `ar()` inside the expression with a pointer to the parameter formulas.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.17 U17: Under identity and inverse links a non-positive mean is returned by `posterior_epred()` without a word, and `posterior_predict()` draws NaN

Where

brms: posterior_epred(), posterior_predict(), negbinomial("identity"), Gamma("inverse"), Gamma("identity"), inverse.gaussian("1/mu^2"). Missing validation; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

E06, E02 (#71, #75); divergence D134, D115.

What

Stan rejects a draw whose mean leaves the family's support, but on new rows brms returns that mean as an expected value and draws NaN responses with only R's random-number warning. `Gamma("inverse")` is the Gamma family's default link.

Observed

b_Intercept = 1, b_x = 1, new row x = -2.33 (eta = -1.33): negbinomial identity and Gamma identity `posterior_epred` -1.33, Gamma inverse -0.75188, each with no error or warning; inverse.gaussian 1/mu^2 NaN (R's "NaNs produced"); `posterior_predict` NaN or NA with R's "NAs produced" / "NaNs produced".

Expected

An error, or NA with a warning naming the rows whose mean is outside the family's support (mu > 0; eta > 0 under 1/mu^2).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U17.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U17.out`.

Fix

Check the predicted mean against the family's support in `posterior_epred()` and `posterior_predict()` (mu > 0 for the negative binomial, Gamma and inverse Gaussian; eta > 0 for the 1/mu^2 link) and stop, or return NA with a warning that names the rows, as for any other invalid input.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.18 U18: asym_laplace accepts a fixed quantile of 0 or 1, where the density is 0 everywhere

Where

brms: asym_laplace(), bf(..., quantile =). Missing validation; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

E02 (#75); divergence D115.

What

brms refuses a fixed quantile outside [0, 1] but accepts the end points, where the asymmetric Laplace density `q (1 - q) / sigma * exp(...)` is 0 for every response, so the model has log density -Inf everywhere.

Observed

`make_standata(bf(y ~ 1, quantile = 0))` and `quantile = 1`: accepted (1.5: refused, "Parameter 'quantile' must be between 0 and 1."); `dasym_laplace(., quantile = 0)` = 0, 0, 0; `log_prob` of the compiled model: -Inf.

Expected

Refuse quantile = 0 and 1 as well: the parameter must lie strictly between 0 and 1.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U18.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U18.out`.

Fix

Require `0 < quantile < 1` for a fixed quantile (the open interval), with the same message.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.19 U19: The default length-scale prior of `gp()` is solved to a tolerance coarser than the 6 decimals it is printed and written with

Where

brms: gp() default lscale prior, get_prior(), default_prior(). Inexact numerics; reproduced. Versions: brms 2.23.0, R 4.5.2.

Found by

C03 (#51); divergence D67. Rounding only: the prior differs from the correctly rounded one in the 6th decimal.

What

The default `inv_gamma(a, b)` puts 1% of the mass below the smallest scaled distance between distinct inputs and 1% above the largest; brms's solver stops within about 1e-7 of the solution, which crosses a rounding boundary at some inputs.

Observed

x = (0, 0.5, 1): `inv_gamma(45.631904, 31.399697)`; x = (0, 0.48188060200668892, 1): `inv_gamma(41.190626, 27.743173)`. (x = (0, 0.3, 1) agrees.)

Expected

The solutions 45.631904608850, 31.399697537363 and 41.190626321252, 27.743173538338 round to `inv_gamma(45.631905, 31.399698)` and `inv_gamma(41.190626, 27.743174)`.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U19.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U19.out`.

Fix

Solve the one equation in the shape to double precision before rounding: with `1 / l ~ Gamma(a, rate b)`, `b = hi * qgamma(0.01, a)` and `pgamma(hi * qgamma(0.01, a) / lo, a) = 0.99` (bracketed root finding to about 1e-14), or write the unrounded values into the model.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.20 U20: Approximate GP `gp(x, k = )`: NaN gradient wherever the spectral density underflows (every lscale above about 1 with k = 30)

Where

brms: gp(x, k = , c =) (generated Stan code: sqrt of the spectral density). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

C03 (#51); divergence D66. The divergence row records this as not probed; reproduced here.

What

Basis function j is weighted by `sqrt(S(w_j))`, `S(w) = sdgp^2 sqrt(2 pi) lscale exp(-lscale^2 w^2 / 2)`. Where S underflows to 0 (`lscale * w_j` above about 38.6) the log density is finite but the chain rule through `sqrt` at 0 gives Inf times 0 = NaN in the gradient of sdgp and lscale. With k = 30 and c = 5/4 that is every lscale above 1.024 on brms's scaled inputs, an ordinary length-scale for a smooth function, where

HMC cannot move.

Observed

`y ~ gp(x, k = 30, c = 5/4)`, 12 rows: at lscale = 1.2 and 2 the log density is finite (-58.82, -62.71) and the gradient in (log sdgp, log lscale) is (NaN, NaN), 2 of the 34 entries; at lscale = 0.5 all entries agree with a finite difference to 7e-9.

Expected

The finite gradient (4.3837865, -4.6773439) at lscale = 1.2 and (1.9224068, -9.2924243) at 2 (central finite differences of brms's own `log_prob`; the other 32 entries agree to 7e-9).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U20.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U20.out`.

Fix

Form the weight without a square root of an underflowed number: `sqrt(S(w)) = sdgp (2 pi)^(1/4) sqrt(lscale) exp(-lscale^2 w^2 / 4)` (or `exp(0.5 * log S)` with `log S` computed directly), whose gradient is finite (and 0 in the limit) where S underflows.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found in brms's NEWS or the issue tracker (read-only searches, 2026-10-21 to 2026-10-23).

2.21 U21: Priors on probability parameters (zi, hu, zoi, coi) are renormalised on (0, 1): divergence row D29 is not what brms does

Where

brms: zero_inflated_poisson() zi prior, zero_one_inflated_beta() zoi and coi priors (lprior). Wrong result; not a bug after all. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

B06 (#39); divergence D29. Checked for this log; the row's statement was not a brms output.

What

D29 says a prior on `zi` / `hu` is added as written, not renormalised on (0, 1). brms 2.23.0's `lprior` divides each such prior by its mass on (0, 1), as it does for the lower-bounded classes, so there is nothing to report upstream. Kept so the row is not reported by mistake.

Observed

zi ~ normal(0.5, 1) at 0.3 with the Intercept at 0.2: lprior -2.50819864514782; zoi ~ normal(0.3, 0.1), coi ~ logistic(0, 1), phi ~ exponential(1): -3.27907096338989.

Expected

Renormalised on (0, 1): -2.50819864514782 and -3.27907096338989, equal (as written they would be -3.46811497884345 and -4.74550578682117).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U21.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U21.out`.

Fix

None in brms. Divergence row D29 now says that brms renormalises these priors on (0, 1).

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not applicable (not a bug).

2.22 U22: Stan's `normal_lccdf` loses its value in the upper tail (-Inf from z of about 8.3), so brms's right-censored rows are -Inf and truncated rows +Inf

Where

Stan Math: normal_lccdf, brms: gaussian() with cens() (right-censored rows) and trunc(lb =). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

B02 (#40); divergence D24, D25. The value loss beyond z of about 5 and the -Inf from 8.3 were measured for this log; the divergence rows record the gradient (U23).

What

`normal_lccdf(z | 0, 1)` = log(1 - Phi(z)) is off by 2.6e-12 relative at z = 5, 2.1e-9 at 6.17, 5.3e-4 at 8 and -Inf from about 8.3, where the value is an ordinary number. brms calls it for a right-censored Gaussian row and for a truncation's lower bound, so such a row 8.3 SD above its mean has log density -Inf, and a row of `y | trunc(lb = L)` with L 9 SD above the mean has log density +Inf (its density minus -Inf).

Observed

Right-censored rows at mu = 0, sigma = 1: z = 5 -15.0649983940277, z = 8 -35.03205247743868, z = 8.3, 9 and 20 -Inf. Truncated rows y = L + 0.5 of `y | trunc(lb = L)`: L = 6.17 -1.365568612357809, L = 8 -2.011886055765999, L = 9 +Inf.

Expected

R's `pnorm(z, lower.tail = FALSE, log.p = TRUE)`: -15.06499839398873, -35.01343715991455, -37.49421742374825, -43.62814911333211, -203.9171553710973; truncated rows -1.365568567117943, -2.030501373290122, -2.41578941987256. Controls (z = 0, 1; L = 0, 1) agree to 1e-15.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U22.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U22.out`.

Fix

Evaluate the upper tail through the lower one, `normal_lccdf(y | mu, sigma) = std_normal_lcdf(-(y - mu) / sigma)` (Stan's `std_normal_lcdf` is exact in value to -40 in these probes), or from `log(erfc(z / sqrt 2) / 2)` with `erfc` (or `erfcx`) rather than `1 - erf`. brms can do the first today for any Stan version.

Belongs in

https://github.com/stan-dev/math

Known upstream

Known and fixed in Stan Math 5.4.0.

https://github.com/stan-dev/math/issues/1985 (closed 2026-08-30): Accuracy of normal_lccdf much worse than normal_lcdf

https://github.com/stan-dev/math/pull/3363 (merged 2026-08-30): Aligned normal_lccdf and std_normal_lccdf with the lcdf functions

https://github.com/stan-dev/math/issues/2847 (closed 2026-10-07): Errors when right-censoring: bug in normal_lccdf?

Fixed in: Stan Math 5.4.0 (released 2026-09-16), which rstan 2.32.7 with StanHeaders 2.32.10 does not include; which StanHeaders release ships it was not checked.

brms issue #1956 (open) notes that `normal_lccdf` saturates earlier than `normal_lcdf` in its discussion of two-sided truncation.

2.23 U23: Stan's `normal_lcdf` / `std_normal_lcdf` derivative is inexact (up to 2.3e-5 relative), and so are brms's censored Gaussian gradients

Where

Stan Math: normal_lcdf, std_normal_lcdf, normal_lccdf (upper tail), brms: gaussian() with cens() and trunc(). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

B02 (#40); divergence D24, D25. `gauss_censored` and `gauss_truncated` are accepted against 50-digit references for their gradients (P6, #47).

What

The value of log Phi(z) is accurate, but its derivative is not the inverse Mills ratio phi(z) / Phi(z) to double precision: it is off by 7e-7 at z = 1, 6.4e-6 at 2, 2.3e-5 at -10 and 1.3e-5 at -38, as if from a separate approximation; `normal_lccdf`'s derivative is off by 4.5e-8 at z = 6.17 and 1.9% at 8. brms's left- and right-censored (and truncated) Gaussian rows differentiate through them.

Observed

Gradient in (mu, log sigma) of one censored row at mu = 0, sigma = 1: left-censored z = -10: (-10.0978573856826, 100.978573856826); z = 2: (-0.0552482144761343, -0.1104964289522686); right-censored z = 8: (8.273965879706052, 66.19172703764842).

Expected

(-10.09809323396249, 100.9809323396249), (-0.05524786267898995, -0.1104957253579799) and (8.121368112236123, 64.97094489788898), from `m(z) = exp(dnorm(z, log = TRUE) - pnorm(z, log.p = TRUE))`; the right-censored row at z = 0 agrees to 1.4e-16. On the frozen cases: up to 2.5e-6 (`gauss_censored`) and 4.2e-7 (`gauss_truncated`) relative.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U23.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U23.out`.

Fix

Differentiate the accurate value instead of an approximation: `d/dz log Phi(z) = phi(z) / Phi(z) = sqrt(2 / pi) / erfcx(-z / sqrt 2)` (with `erfcx`, now a Stan Math function), or `exp(log phi(z) - log Phi(z))` from the same log CDF the value uses.

Belongs in

https://github.com/stan-dev/math

Known upstream

Not found as a report of the derivative's accuracy; an open pull request reworks std_normal_lcdf.
https://github.com/stan-dev/math/pull/3406 (open PR): Expanded numerical stability and performance for `std_normal_lcdf`, reused across distributions & functions
https://github.com/stan-dev/math/issues/2881 (open): normal_lcdf and normal_lccdf give infinite gradients for infinite inputs.
Whether PR #3406 makes the derivative exact was not checked (its code was not read).

2.24 U24: cumulative("probit") through Stan's `ordered_probit_lpmf`: exact log density, gradient off by up to 1.2e-5 relative

Where

Stan Math: ordered_probit_lpmf, brms: cumulative(link = "probit"). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

D04 (#64); divergence D88. Accepted with a 50-digit reference for the gradient (P6, #66).

What

brms writes the cumulative probit likelihood with Stan's `ordered_probit_lpmf` (the generated program calls it). Its value equals the exact log likelihood to 2e-16, but its gradient in the thresholds does not; size and pattern match U23's `std_normal_lcdf` derivative, which Stan issue #2228 says `ordered_probit` uses (a likely common cause, not verified from Stan's code).

Observed

One row per category, thresholds (-2, 0.5, 2.5): gradient (-1.1103089658908, -8.506981885742, -5.5295205825131), 1.2e-5 relative off; (-1, 1, 2): 2.4e-6; (-6, -4.5, 0.3): 7.2e-6. On the frozen case `cumulative_probit_varint`: up to 6.5e-5, at 173 of 176 entries.

Expected

(-1.1103226460712, -8.5069986883767, -5.5295340249966), from `phi(t_k) / P` and `-phi(t_{k-1}) / P` through Stan's ordered transform; the log densities (-10.46347710031321, -8.001719394174177, -34.77299757782713) agree to 2e-16.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U24.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U24.out`.

Fix

Differentiate `log(Phi(b) - Phi(a))` exactly: `d/db = phi(b) / (Phi(b) - Phi(a))`, `d/da = -phi(a) / (Phi(b) - Phi(a))`, each ratio as `exp(log phi - log mass)` with the mass from the tail both arguments lie in; with U23 fixed (an exact derivative of `std_normal_lcdf`) this may follow.

Belongs in

https://github.com/stan-dev/math

Known upstream

Not found as such; related work on std_normal_lcdf is open.

https://github.com/stan-dev/math/issues/2228 (closed 2020-12-09): Use std_normal_lc(c)df functions for the ordered_probit distribution

https://github.com/stan-dev/math/pull/3406 (open PR): Expanded numerical stability and performance for `std_normal_lcdf`, reused across distributions & functions

2.25 U25: Stan's `gamma_lccdf` derivative in the shape is inexact (up to 1.2e-2 relative), and so are brms's right-censored Gamma gradients

Where

Stan Math: gamma_lccdf, brms: Gamma() with cens() (right-censored rows). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

E04 (#77); divergence D120. `gamma_censored_interval` is accepted with a 50-digit reference for its gradient (P6, #78).

What

The value of log Q(shape, x) is right, and so is `gamma_lcdf`'s shape derivative (to 1e-10), but `gamma_lccdf`'s shape derivative is not: brms's right-censored Gamma rows (`gamma_lccdf`, rate = shape / mu) have inexact gradients in log(shape). brms's interval rows use `gamma_lcdf` and are accurate here.

Observed

Gradient in (log mu, log shape) of one right-censored row: y = 7, mu = 5, shape = 5: (3.691412255656, -0.4868664317708), 1.2e-2 relative off in log(shape); y = 2, mu = 5, shape = 10: 2.5e-4; y = 3, mu = 2, shape = 2: 6.1e-5. On the frozen case `gamma_censored_interval`: up to 2.8e-5.

Expected

(3.691412255645, -0.4809855583225), 0.02945472493298 and -0.2673022227384 in log(shape), from fourth-order differences of R's `pgamma(., lower.tail = FALSE, log.p = TRUE)` (accurate to about 1e-11); the left-censored and interval rows agree to 1e-10.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U25.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U25.out`.

Fix

Compute the shape derivative of log Q(a, x) to double precision, e.g. from the derivative of log P (series for $x < a + 1$, `dQ/da = -dP/da`) and the continued fraction of Q differentiated through its recurrence above, iterated to machine precision; or by quadrature of `int_x^inf log(t) t^(a - 1) exp(-t) dt`.

Belongs in

https://github.com/stan-dev/math

Known upstream

Known: the incomplete gamma gradients are an open Stan Math issue, with work merged since StanHeaders 2.32 and an open pull request.

https://github.com/stan-dev/math/issues/2006 (open): gamma_p and gamma_q gradient problems

https://github.com/stan-dev/math/pull/3408 (open PR): Incomplete gamma gradient roots and log-space cdfs for six distributions

https://github.com/stan-dev/math/issues/3257 (closed 2026-04-16): Fix `gamma_lccdf` actually
https://github.com/stan-dev/math/pull/3275 (merged 2026-04-16): Fix gamma lccdf
Whether the gamma_lccdf rework merged in 2026 (first in Stan Math 5.3.0) makes the shape derivative exact was not checked; nothing newer than StanHeaders 2.32.10 could be installed.

2.26 U26: Stan's `von_mises_lpdf` overflows (`cyl_bessel_i` numeric overflow) for kappa above about 713.9

Where

Stan Math: von_mises_lpdf, brms: von_mises(). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

E02 (#75); divergence D113.

What

The density's normalising constant I0(kappa) is evaluated itself and overflows, so `log_prob` raises an error (and a fit cannot start) for a precision above about 713.9, where the density is ordinary. brms's R-side `dvon_mises` (log_lik) is exact there.

Observed

One row y = 0.1 at mu = 0: kappa = 700 and 713 within 8.4e-14 of the exact value; kappa = 714, 720, 1000: "Exception: Error in function boost::math::cyl_bessel_i<double>(double,double): numeric overflow".

Expected

`kappa (cos(y - mu) - 1) - log(2 pi) - log(i0e(kappa))`: -1.200698236329766, -1.226487658870219, -2.46102067825299 (R's `besselI(kappa, 0, expon.scaled = TRUE)`).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U26.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U26.out`.

Fix

Never form I0(kappa): `log I0(kappa) = kappa + log(i0e(kappa))` with the exponentially scaled Bessel function (or the large-kappa asymptotic series), so the log density is `kappa (cos(y - mu) - 1) - log(2 pi) - log(i0e(kappa))`.

Belongs in

https://github.com/stan-dev/math

Known upstream

Probably fixed upstream: a von_mises_lpdf stability fix merged in 2024 (not verified against this overflow).
https://github.com/stan-dev/math/issues/3035 (closed 2024-03-23): Improve numerical stability of von_mises_lpdf
https://github.com/stan-dev/math/pull/3036 (merged 2024-03-23): Improve numerical stability of von_mises_lpdf
https://github.com/paul-buerkner/brms/issues/1633 (closed 2024-08-13): use von_mises_lpdf directly instead of von_mises2_lpdf after next stan update
Fixed in: probably Stan Math 4.9.0 (2024-06-03), later than the Stan Math in StanHeaders 2.32.10; not verified, since nothing newer could be installed.

2.27 U27: Stan's `skew_normal_lpdf` and `exp_mod_normal_lpdf` are -Inf once their normal-CDF factor underflows (argument below about -37.5)

Where

Stan Math: skew_normal_lpdf, exp_mod_normal_lpdf, brms: skew_normal(), brms: exgaussian(). Inexact numerics; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

D01 (#60); divergence D77. D01's probes used rstan::expose_stan_functions; here the same comes from brms models.

What

Both densities contain a factor log Phi(x) that is evaluated as the log of a CDF value; it is -Inf once that value underflows: a skew-normal row far on its short side, an ex-Gaussian row far below its mean.

Observed

skew_normal mu = 0, sigma = 1, alpha = 10: y = -5 exact (-259.5392660824244), y = -8 and -9 -Inf. exgaussian mu = 0, sigma = 1, beta = 1: y = -36 exact (-617.0032275937984), y = -40 -Inf.

Expected

-841.9599722745395 and -1110.717630363651 (skew normal), -765.1084420137538 (ex-Gaussian), with log Phi from R's `pnorm(x, log.p = TRUE)`; the control rows (y = 0.5) agree to 7e-16.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U27.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U27.out`.

Fix

Evaluate the factor with an accurate log CDF: `std_normal_lcdf(x)`, or `log(erfcx(-x / sqrt 2) / 2) - x^2 / 2` for x < 0, instead of the log of `Phi` or `erfc`.

Belongs in

https://github.com/stan-dev/math

Known upstream

Not found for these log densities' lower tail; related work is open.

https://github.com/stan-dev/math/pull/3406 (open PR): Expanded numerical stability and performance for `std_normal_lcdf`, reused across distributions & functions

https://github.com/stan-dev/math/pull/3443 (open PR): Compute skew_normal_lcdf and skew_normal_lccdf without cancellation in the tails

https://github.com/stan-dev/math/issues/2803 (closed 2026-10-06): Fix Exponentially modified Gaussian overflows at lambda >= 40

#3443 is about the CDFs and #2803 about another regime (large lambda), not these log densities' lower tail.

2.28 U28: loo's `stacking_weights()` stops short of the optimum (weights off by up to 3.7e-4, no exact zeros)

Where

loo: stacking_weights(), loo_model_weights(method = "stacking"), brms: model_weights(weights = "stacking"). Inexact numerics; reproduced. Versions: loo 2.9.0, R 4.5.2.

Found by

E05 (#73); divergence D126. Stacking is withheld in AutoBayes pending a decision; on the frozen case `gauss_loo_compare_x_stacking` brms's weights are 1.04e-8 from the optimum.

What

The stacking objective is maximised by `stats::constrOptim` (a log-barrier method) and the result returned at its tolerance, so the weights are near, not at, the optimum, and a model the optimum excludes keeps a small positive weight.

Observed

Seeded pointwise elpd sets: `two_nearly_equal` loo (0.999630903495, 0.000369096505); `five_200` up to 7e-5 off with 1.5e-7 on an excluded model; `four_one_dominated` 9.4e-6 off with 1.7e-8 on the excluded model; objective up to 2e-6 below the optimum's.

Expected

(1, 0) for `two_nearly_equal`, exact zeros for excluded models, and the KKT optimum of `sum_i log(sum_k w_k exp(lpd_ik))` on the simplex (computed in the script by multiplicative updates and active-set Newton steps).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U28.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U28.out`.

Fix

Solve the stacking problem to its KKT conditions (`g_k = sum_i exp(lpd_ik) / p_i` equal to N on the support and at most N off it), e.g. by an active-set Newton method on the simplex, which gives exact zeros.

Belongs in

https://github.com/stan-dev/loo

Known upstream

Known: an open issue to improve the optimiser and an open pull request with an active-set Newton method.
https://github.com/stan-dev/loo/issues/119 (open): accelerate the optimization algorithm in stacking
https://github.com/stan-dev/loo/pull/412 (open PR): Compute stacking weights with an active-set Newton method
loo 2.10.1 is on CRAN; it was not tested here (loo 2.9.0 installed).

2.29 U29: `loo_compare()` compares a PSIS-LOO result with a WAIC result without an error or a warning

Where

loo: loo_compare(). Missing validation; reproduced. Versions: loo 2.9.0, R 4.5.2.

Found by

D05 (#58); divergence D93.

What

A `waic` object is also of class `loo`, and `loo_compare()` accepts it next to a `psis_loo` object, naming the table's columns after the first argument's criterion, so a mixed table reads as a LOO comparison.

Observed

`loo_compare(loo(ll1), waic(ll2))` returns a table with columns `elpd_loo`, `p_loo`, `looic` and no warning.

Expected

An error, or at least a warning, that the two results use different criteria.

Reproduce

`Rscript docs/brms-parity/upstream/repro/U29.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U29.out`.

Fix

Check that all compared objects come from the same criterion (e.g. all `psis_loo`, all `waic`, all `kfold`) and refuse or warn otherwise.

Belongs in

https://github.com/stan-dev/loo

Known upstream

Not found in loo's NEWS or issue tracker (read-only searches).
loo 2.10.0 changed `loo_compare()`'s output (#300); whether it now checks the criteria was not tested (loo 2.9.0 installed).

2.30 U30: loo's MCSE of an observation's elpd_loo is NaN once that elpd is below about -372

Where

loo: loo(), mcse_elpd_loo (pointwise and total). Inexact numerics; reproduced. Versions: loo 2.9.0, R 4.5.2.

Found by

C07 (#49); divergence D50.

What

The Monte Carlo standard error of elpd_loo_i is NaN once its square on the probability scale underflows, and the total MCSE with it; elpd_loo, p_loo and k-hat are unaffected. Shifting the observation's log-likelihood by a constant shifts elpd_loo_i and leaves every other per-observation quantity unchanged, so its MCSE should not change either.

Observed

1000 draws x 5 observations, the fifth near -400: its `mcse_elpd_loo` is NaN and `print()` says "MCSE of elpd_loo is NaN."

Expected

0.0134962534892, the MCSE of the same column shifted by +400 (whose elpd_loo, -0.0836882186983, is the original's plus 400, and whose p_loo and k-hat are identical).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U30.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U30.out`.

Fix

Evaluate the MCSE after subtracting `max_s log p_is` from the observation's log-likelihood (the MCSE is invariant to that shift), or carry the log-normal approximation's terms relative to `exp(elpd_loo_i)`, so that nothing squares `exp(elpd_loo_i)`.

Belongs in

https://github.com/stan-dev/loo

Known upstream

Not found as such; an open issue on the MCSE and a merged accuracy change in the development version may cover it.

https://github.com/stan-dev/loo/issues/245 (open): Improved mcse_elpd_loo

https://github.com/stan-dev/loo/pull/395 (merged 2026-09-15): Improve floating point accuracy

PR #395 is in loo's development version only (CRAN has 2.10.1); whether it fixes this was not tested.

2.31 U31: A one-row model with `trunc()` (a per-row or a constant bound) cannot be built: rstan stops at data initialisation

Where

brms: trunc(lb =), brm(), make_standata(). Wrong result; reproduced. Versions: brms 2.23.0, rstan 2.32.7, StanHeaders 2.32.10, R 4.5.2.

Found by

upstream log (U22). Found while writing U22, whose truncated rows use two identical rows for this reason.

What

With a single data row, `make_standata()` gives the truncation bound `lb` as a plain number (no dim) while the program declares it with length N = 1, so rstan refuses the data ("mismatch in number dimensions declared and found in context ... variable name=lb; dims declared=(1); dims found=()") and `brm()` returns an object without a valid model. Rare in practice, but it breaks one-row checks and examples.

Observed

`y | trunc(lb = L) ~ 1` and `y | trunc(lb = 1) ~ 1` with one row (y = 1.5, L = 1): the rstan error above at `brm(..., chains = 0)`, then "the model object is not created or not valid" from `log_prob`.

Expected

A model whose log likelihood at mu = 0, sigma = 1 is `log phi(1.5) - log(1 - Phi(1))` = -0.2029168881954091, as the same model with two identical rows gives twice that (brms -0.4058337763908186) and a one-row model without truncation gives `log phi(1.5)` (brms -2.043938533204672, exact).

Reproduce

`Rscript docs/brms-parity/upstream/repro/U31.R` from the repository root; it exits with status 1 while the problem is present. Output of one run: `docs/brms-parity/upstream/repro/U31.out`.

Fix

Pass the bounds (and any other per-row data entry) to Stan as arrays of length N even when N = 1, e.g. with `as.array()`, as brms already does for the response.

Belongs in

https://github.com/paul-buerkner/brms

Known upstream

Not found for truncation bounds; the same kind of one-row failure was reported for an offset in 2018.

https://github.com/paul-buerkner/brms/issues/545 (closed 2018-11-06): Can't sample when using offset with a single data point